GridCast: Design Document & Implementation Plan

Amendment to the Project Proposal | Draft for Team Review

[Author names omitted for review]

Abstract

This document extends the original GRIDCAST proposal with three additions: (1) corrections and clarifications to the original proposal where assumptions required revision; (2) a concrete user story defining who interacts with the system and how; and (3) a full implementation design covering the machine learning architecture, technology stack, infrastructure plan, and phased build sequence. The target artefact is a live, interactive hackathon demo deployable on IBM Cloud within a $100 budget, targeting the IBM Technologies integration prize.

1 Amendments to the Original Proposal

The original proposal is technically sound in its framing and data sourcing strategy. Three points require clarification or amendment before implementation begins.

1.1 GenCast Compute Requirements

The original proposal presents GENCAST as a freely runnable inference layer. In practice, GENCAST requires TPU infrastructure to generate the 50-member ensemble at meaningful scale. At cloud GPU rates (approximately $3–6 per run on an A100), a handful of demonstration runs are feasible within budget, but continuous inference is not.

Demo resolution: The live demonstration will use the **Open-Meteo Ensemble API** as the probabilistic weather source. Open-Meteo provides a 16-member GFS ensemble at no cost, no API key required, with a 10-day forecast horizon at hourly resolution — sufficient to drive the probabilistic pipeline. In the pitch and accompanying materials, GENCAST is presented as the production upgrade path with full narrative continuity.

1.2 ML Model Specification

The original proposal describes a "weather-to-grid-stress mapping" without specifying the model architecture. This is the core ML contribution and requires a concrete choice.

Selected model: A **Temporal Fusion Transformer** (TFT), implemented via the `pytorch-forecasting` library. The TFT was chosen for three reasons: it handles multi-horizon forecasting natively (up to 240 hours / 10 days), it accepts mixed tabular and temporal inputs matching the three-layer feature matrix, and it produces native output quantiles without post-hoc calibration. The quantile outputs directly populate the fan chart bands in the user interface.

1.3 Live Inference vs. Backtesting

The original proposal conflates validation (backtesting on historical stress events) with real-time operation (hourly contract triggers). These are distinct modes.

Clarification: The model is *trained offline* on historical data. "Live" in the demo context means: at demo time, the system fetches today's real EIA demand data and today's real Open-Meteo ensemble forecast, runs inference against the trained model, and updates the map in real time. The model is not re-trained live. The three historical stress events (Texas 2021, Pacific Northwest 2022, PJM Summer 2023) serve as a *stress event replay* mode in the UI, showing what the forecast would have looked like 10 days before each event occurred.

2 User Story

2.1 Product Tiers

GRIDCAST is designed in two tiers with distinct users.

Tier 1 — Demo & Investor View (hackathon target). The audience is investors, judges, and potential utility partners evaluating the concept. The interface is a public US map showing the transmission nodes under observation, colour-coded by current stress probability. A visitor clicks a node and immediately sees a 10-day probabilistic forecast, the current recommended power allocation percentage, and a set of simulated data centre neighbours with their assigned draw percentages. This tier uses exclusively public data and requires no access agreement with any utility.

Tier 2 — Operator-Facing Dashboard (production path). Once a utility or grid operator licenses the product and provides internal SCADA or settlement data, the interface shifts from the public map to an internal operations view showing the utility's own grid topology, real transmission node identifiers, live metering data, and contractual commitment windows. The operator persona is a grid reliability engineer or demand response manager who runs a 10-day forecast each morning and uses it to decide which flexible contracts to trigger, how much capacity to commit to data centre tenants, and when to issue curtailment notices.

2.2 Core User Interaction — Tier 1

The canonical user action for the demo proceeds as follows:

1. The user opens the web application and sees a full US map with four highlighted transmission nodes.
2. Each node displays a colour indicator (green / amber / red) driven by the current 24-hour peak stress probability from the model.
3. The user clicks the **Dominion Hub** node (Northern Virginia).
4. A side panel opens showing: the 10-day forecast fan chart, the current LMP congestion value, the EIA demand deviation (actual vs. forecast), and the headline recommended allocation percentage with its confidence band.
5. The panel also shows three simulated data centres nearby, each assigned a draw percentage for today derived from the allocation output.
6. The user clicks **"Refresh Forecast"**. The backend fetches the latest EIA hourly reading and the latest Open-Meteo ensemble, reruns inference, and the chart updates visibly.
7. The user selects **"Replay: Texas Winter Storm 2021"** from a dropdown. The map transitions to a replay view showing the forecast as it would have appeared on 1 February 2021 — 10 days before the storm peak. The fan chart shows the growing uncertainty band as the horizon extends, illustrating exactly the utility of confidence bounds.

3 Target Nodes

Four transmission nodes are selected for the demonstration, covering three ISOs and two California zones. Table 1 lists the nodes, their rationale, and their data sources.

Table 1. Target transmission nodes for the GridCast demo.

Node	ISO	State	EIA BA	Narrative
Dominion Hub	PJM	Virginia	PJM	Largest US data centre corridor; \$38.70/MWh congestion case in proposal Appendix A
CAISO SP15	CAISO	California	CISO	Southern California / LA area; duck curve stress window (15:00–18:00 solar ramp)
CAISO NP15	CAISO	California	CISO	Northern California / Bay Area; tech hub density, distinct topology from SP15
ERCOT Houston	ERCOT	Texas	ERCO	Site of 2021 Winter Storm Uri; cold-snap demand surge validation case

4 Technical Architecture

4.1 Revised Three-Layer Pipeline

The three-layer structure from the original proposal is preserved. Layer 1 is amended to substitute Open-Meteo for live GENCAST inference; the ML model in Layer 3 is now specified as a TFT.

Layer 1 — Probabilistic Weather (Open-Meteo / GRAPHCAST + GENCAST). During training, ERA5 reanalysis provides the ground-truth weather input. At inference time, the Open-Meteo Ensemble API provides a 16-member GFS ensemble for the 10-day horizon. In production, this layer is replaced by GENCAST inference for higher resolution and accuracy.

Layer 2 — Regional Grid Context (EIA API). Unchanged from the original proposal. Hourly demand, forecast demand, and generation by fuel type from EIA Form 930, keyed by Balancing Authority.

Layer 3 — Node-Level Stress (Grid Status LMP + TFT). Historical nodal LMP (congestion component) trains the TFT alongside the weather and EIA features. At inference time, the day-ahead LMP from Grid Status provides a near-term forward signal that anchors the encoder window.

4.2 Temporal Fusion Transformer Configuration

Table 2. TFT model configuration.

Parameter	Value
Library	`pytorch-forecasting`
Encoder length	168 h (1 week of past context)
Decoder length	240 h (10-day forecast horizon)
Output quantiles	0.10, 0.25, 0.50, 0.75, 0.90
Target variable	LMP congestion component (\$/MWh)
Known future inputs	Hour, day-of-week, month (cyclically encoded); Open-Meteo ensemble temperature and wind
Unknown past inputs	EIA actual demand; EIA demand deviation; renewable penetration; LMP congestion z-score
Training period	2020–2022
Validation period	2023
Test period	2024

4.3 Headline Output Formula

The model output is operationalised as a single recommended allocation percentage for consumption by the UI:

$$\text{AllocationPct} = \left(1 - P_{90\%}^{\text{stress}}\right) \times 100$$

where $P_{90\%}^{\text{stress}}$ is the fraction of the 10-day horizon for which the 90th-percentile quantile forecast exceeds the node's historical stress threshold (defined as the 90th percentile of the LMP congestion component over the training period). This gives a conservative, interpretable single number: if the worst plausible scenario keeps the node below threshold for the full horizon, the operator can commit 100% of excess capacity.

5 Infrastructure and Hosting

5.1 IBM Cloud Stack

The deployment targets IBM Cloud to qualify for the hackathon IBM Technologies integration prize. All components fall within the IBM Cloud free trial tier.

Table 3. IBM Cloud service mapping.

Component	IBM Service	Role
Model artefacts & processed data	IBM Cloud Object Storage	Stores serialised TFT weights (`.ckpt`) and preprocessed Parquet feature files
Backend API	IBM Cloud Code Engine	Serverless container hosting the FastAPI application; loads model weights from COS on startup
Model serving (production path)	watsonx.ai deployment space	Registers and serves the TFT as a Watson ML endpoint; demonstrated in the pitch as the production serving layer
Container registry	IBM Container Registry	Stores the Docker image for the Code Engine application
Frontend	Vercel (external)	Static React application; zero-cost, instant deployment

5.2 Architecture Diagram

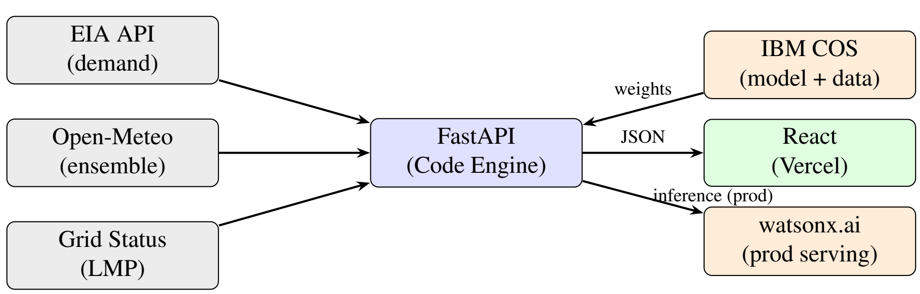

Figure 1. GridCast system architecture. Arrows show data flow at inference time.

5.3 API Endpoints

Table 4. Backend API surface.

Method	Path	Description
GET	`/nodes`	List of 4 nodes with current stress colour and allocation %
GET	`/nodes/{id}/forecast`	10-day quantile forecast as JSON (5 quantiles $\times$ 240 hours)
GET	`/nodes/{id}/live`	Latest EIA snapshot + Open-Meteo ensemble summary
POST	`/nodes/{id}/refresh`	Pull fresh data and rerun inference; returns updated forecast
GET	`/replay/{event_id}`	Pre-computed backtest output for a named historical stress event

6 Build Sequence

The build is organised into seven phases. Phases 1a–1d and Phase 6 can run in parallel once Phase 0 is complete. Phase 3 (training) is the longest serial step and blocks Phase 4.

Table 5. Phased build sequence with dependencies.

Phase	Name	Tasks	Blocks
0	Accounts & Keys	IBM Cloud free trial; IBM COS bucket; Code Engine project; watsonx.ai project; EIA API key; Mapbox token; CDS account	Everything
1a	ERA5 download	`cdsapi`: 2 m temp, 10 m wind, solar radiation; bounding boxes around 4 nodes; 2020–2024; upload to COS	Phase 2
1b	EIA pipeline	`requests`: PJM / CISO / ERCO demand + fuel-type data 2020–2024; upload to COS	Phase 2
1c	LMP pipeline	`gridstatus`: Dominion Hub, SP15, NP15, Houston Hub; real-time + day-ahead 2020–2024; upload to COS	Phase 2
1d	Merge & align	Resample all sources to hourly UTC; join on timestamp + node; output one Parquet per node to COS	Phase 2
2	Feature engineering	Demand deviation; renewable penetration; LMP z-score; solar ramp rate; demand/capacity ratio; cyclical time encoding	Phase 3
3	Model training	TFT on Colab T4 GPU; encoder 168 h, decoder 240 h; quantiles [0.1, 0.25, 0.5, 0.75, 0.9]; validate on 3 stress events; serialise `.ckpt`; upload to COS	Phase 4
4	Backend API	FastAPI; loads weights from COS; calls EIA + Open-Meteo; runs TFT inference; 5 endpoints; Dockerfile	Phase 5
5	IBM deployment	Push image to IBM Container Registry; deploy Code Engine application; set env vars; register model in watsonx.ai deployment space	Phase 7
6	Frontend	React + Mapbox GL JS (map); Recharts (fan chart); node panel with allocation %; data centre overlays; replay mode; deploy on Vercel	Phase 7
7	Integration	Wire frontend to Code Engine URL; test all 4 nodes end-to-end; validate replay for Texas 2021; add IBM badge	Demo

Critical path. Phase 0 $\rightarrow$ {1a, 1b, 1c} in parallel $\rightarrow$ 1d $\rightarrow$ 2 $\rightarrow$ 3 $\rightarrow$ 4 $\rightarrow$ 5 $\rightarrow$ 7. Phase 6 (frontend) can be built with mocked JSON responses while Phases 3–5 are in progress.

7 Technology Stack Summary

Table 6. Full technology stack.

Layer	Technology	Notes
Weather (demo)	Open-Meteo Ensemble API	Free, no key, 16-member GFS ensemble
Weather (prod)	GenCast/ GraphCast	Google DeepMind open weights; requires GPU/TPU
Grid demand	EIA API v2	Free, API key required
Nodal LMP	`gridstatus`	Free tier for recent data
Historical weather	ERA5 via `cdsapi`	Free, CDS account required
ML model	PyTorch Forecasting TFT	Quantile regression, multi-horizon
Training	Google Colab (T4 GPU)	Free tier sufficient for 4-node dataset
Backend	FastAPI + Python 3.11	Containerised via Docker
Hosting	IBM Cloud Code Engine	Serverless; free trial
Storage	IBM Cloud Object Storage	Model artefacts + processed data
Model serving	watsonx.ai	Production path; registered in demo pitch
Container image	IBM Container Registry	Free tier
Frontend	React + Vite	
Map	Mapbox GL JS	Free tier: 50k loads/month
Charts	Recharts	Fan chart for quantile bands
Styling	Tailwind CSS	
Deploy	Vercel	Free tier

A Updated Data Sources

A.1 Open-Meteo Ensemble API

Source: open-meteo.com. Free, no API key required. Provides a 16-member GFS ensemble (`gfs_seamless` model) at hourly resolution up to 16 days ahead. Variables used: `temperature_2m`, `wind_speed_10m`, `shortwave_radiation`. The ensemble spread across 16 members provides the uncertainty signal that propagates through the TFT as known future covariates.

Listing 1. Open-Meteo ensemble query for Dominion Hub (Northern Virginia).

```
GET https://ensemble-api.open-meteo.com/v1/ensemble
  ?latitude=38.9&longitude=-77.0
  &hourly=temperature_2m,wind_speed_10m,shortwave_radiation
  &models=gfs_seamless
  &forecast_days=10
```

A.2 EIA, Grid Status, ERA5

Unchanged from the original proposal Appendix A. EIA API v2 remains the source for regional demand and fuel-mix data. The `gridstatus` Python library remains the source for nodal LMP. ERA5 via `cdsapi` remains the historical weather ground truth for training.